

Joshua Finlayson

JavaScript 101

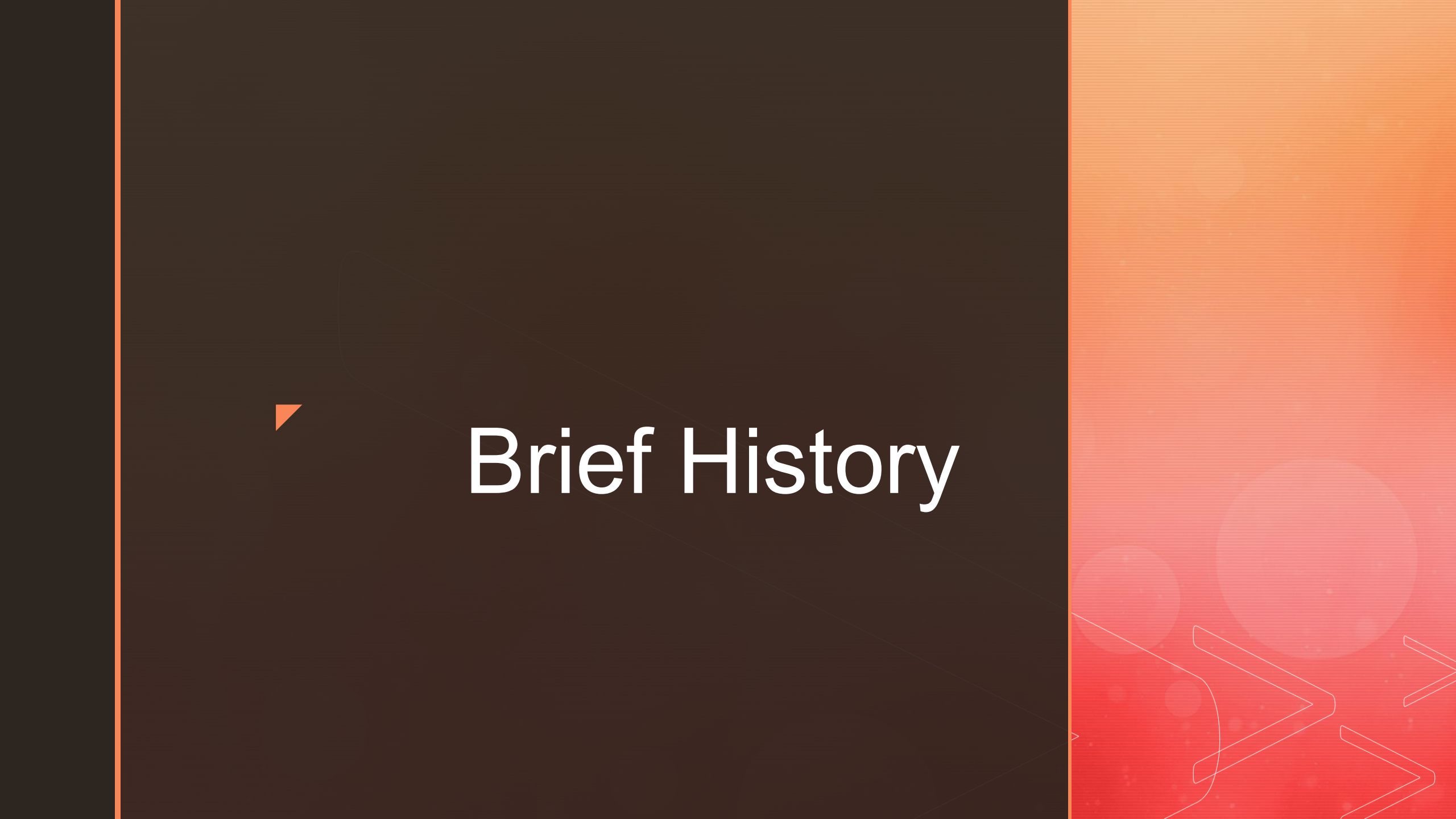


Requirements

1. Clone this Repo: <https://github.com/Raahguu/JS-Workshop-RedRoom-262>
2. Have a Web Browser
3. Have a Text Editor

Overview

1. Intro
2. Brief History
3. Different javascript runtimes
4. Basic code for javascript
5. The DOM
6. Lambda Functions and Arrays
7. Promises



Brief History

The problem

- Before 1995, websites were all static
- All that a website did was query the server and the server would tell the client what to display and how to display it
- Anything more complex would require numerous more requests to the server, and for the server to do everything
- This would bottleneck the website using up large amounts of bandwidth and server compute time

Development

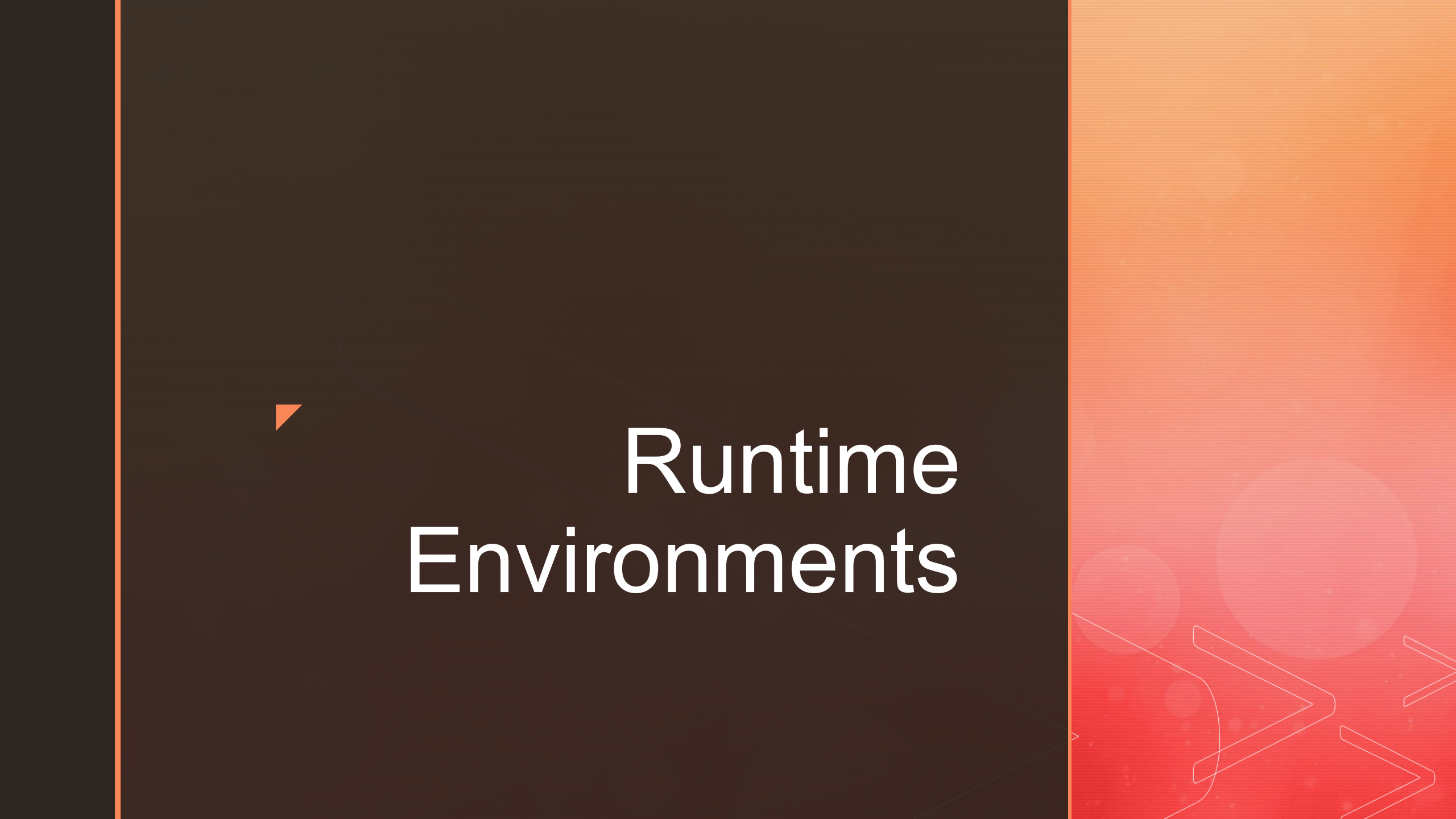
- To fix this, javascript was made to run code on the clients computer
- It was developed in 1995 by Brenden Eich for Netscape navigator
- Eich called it Mocha
- The official name then changed to livescript
- Then the PR guys came in and decided to call it 'javascript' to capitalise on the 'java' craze at the time

ECMAScript

- All the different browser engines (Mosaic, and Trident) implemented javascript in slightly different ways, and so each page had to be programmed twice
- This was horrible, so the javascript syntax was submitted to ECMA to make a standard called 'ECMAScript' in 1997
- Everything now follows this ECMAScript standard which is slowly updated to grow the functionality of javascript over time

What is javascript

- These days javascript is a just-in-time (JIT) compiled dynamically typed language
- 'JIT compiled' means that each function is compiled just before it is run
- This JIT means that functions can be optimised more for what exactly the values passed into them are, instead of optimised generally
- 'dynamically typed' means that variables are declared without specifying their type, and the program dynamically figures out the types



Runtime Environments

V8 Engine

- Most of the time, javascript is running in a web browser
- As such web browsers develop compilers/runtime engines for the language
- Chromium (the most common browser that most things fork off) created the V8 engine in 2008
- V8 also allows web assembly



SpiderMonkey

- Just like how chromium uses V8, firefox uses 'SpiderMonkey'
- SpiderMonkey also does Web Assembly

Node

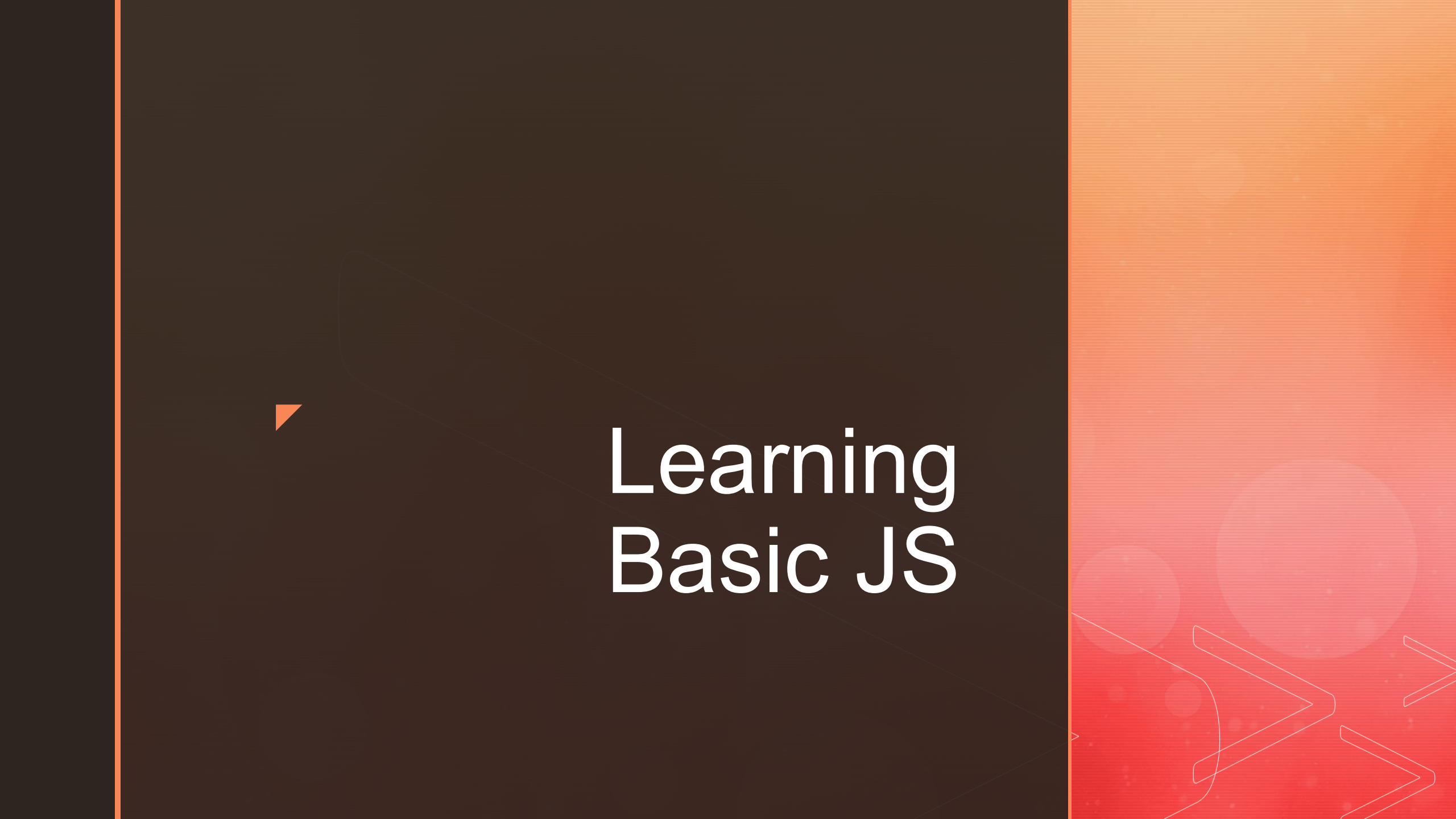
- In 2009 Ryan Dahl decided that he loved javascript, and that he wanted to run it client side as well
- So he ported the V8 engine to be a standalone application and called it 'Node.js'
- There is a package manager for Node.js called 'Node Package Manager' which is shortened to 'npm'

Bun

- A competitor to Node.js was made called 'Bun' in 2021
- Bun is not only a runtime engine, but also its own package manager
- Bun can also natively run and transpile typescript files

wscript & cscript

- There is a legacy application in all windows systems (since 98) called 'wscript' and another called 'cscript'
- These two scripts can run old javascript code, which we call 'JScript'
- The code, while using the same syntax and semantics has different ways of interacting with the os, then normal javascript interacts with the browser



Learning Basic JS

How to run JS code

- Javascript for a website goes into the '`<script>`' tag
- This code can either be located within the '`<script>`' tag, or the '`<script>`' tag can have the 'src' property to import a '.js' file
- This '`<script>`' tag should go at the bottom of the HTML body, to ensure that everything else loads first and can be interacted with

Printing to the console

- Before covering anything else, while using 'inspect element' in a web browser there is the 'console'
- This 'console' can be used to run js code for the webpage, or to print out values from js
- It is designed for debugging a website
- Any error messages will be shown here

Printing to the console cont.

```
1 // This prints 'Yes!' to the console
2 console.log("Yes!");
```

- Code shows some things
 1. All lines of JavaScript must have a semi-colon at the end
 2. Just like in python, strings are done within " " or ' '
 3. There is a global object called 'console' that has the method 'log'
 4. Single line comments are done by tsarting with '//'

Variables

```
1  /*  
2  |   This is a multi-line comment  
3  */  
4  // make a number called 'a'  
5  let a = 8;  
6  // make a number called 'b'  
7  let b = 4.5;  
8  
9  // make a string called 'c'  
10 let c = "Hello";  
11 console.log(a, b, c);
```

- A multi-line comment is done between '/*' and '*/'
- Variables are defined using the 'let' keyword

JS primitive types

- In python there are the primitive types: int, float, string, and boolean, null
- In JavaScript the primitive types are: number, BigInt, string, boolean, null, and undefined
- That's right, both 'integers' and 'floats' are just the 'number' type

consts

```
1  // Set a to be a constant value
2  const a = 8;
3  // print
4  console.log(a);
5
6  // reassign a
7  a = 9; // Error cannot Reassign
8  // print
9  console.log(a);
```

- In javascript, instead of using the 'let' keyword, the 'const' keyword can be used to create a variable which cannot be reassigned

Mutating Consts

```
1  // Set a to be a constant value
2  const a = [1, 2];
3  // print
4  console.log(a);
5
6  // mutate a
7  a.push(3);
8  // print
9  console.log(a);
```

- If the value for the 'const' is a mutable type (e.g. an array (called list in python)), then while the const cannot be reassigned, the underlying value can be mutated

Functions

```
1  function greet(name) {  
2  |    console.log("Hello", name);  
3  |}  
4  
5  greet("Alice");
```

- The function syntax in JavaScript is slightly different than in python
- The word 'function' is used to define it
- And the function is wrapped in '{' and '}' to indicate where it starts and stops instead of the indentation python has

```
1  function check(age) {
2      if (age >= 18) {
3          console.log("R18+");
4      }
5      else if (age >= 15) {
6          console.log("MA15+");
7      }
8      else if (age >= 13) {
9          console.log("PG13+");
10     }
11     else {
12         console.log("G");
13     }
14 }
15
16 check(73);
17 check(18);
18 check(14);
19 check(7);
```

If statements

- If statements require the word 'if', then in brackets, any statement which results in a bool
- then '{' and '}' to encase the conditional code
- 'else if' can be used afterwards
- Then 'else'

Operators

Operator	Use	Example
+	Addition	$3 + 2 == 5$
-	Subtraction	$3 - 2 == 1$
*	Multiplication	$3 * 2 == 6$
/	Division	$3 / 2 == 1.5$
%	Modulus	$3 \% 2 == 1$
**	Power	$3 ** 2 == 9$
+	Concatenate	"N" + "o" == "No"

* A source of much confusion, is if a '+' operator is used between a number and a string, then the number is converted into a string, and concatenation is done, not addition

Increment/Decrement

```
1 // set a to 7
2 let a = 7;
3
4 // a is now 8
5 a++;
6
7 console.log(a);
```

- The operators '++' and '--' used on numbers increases/decreases the value of the variable they are used on by 1

Comparison Operators

Operator	Description	Example	Example Result
==	Equal to	"2" == 2	true
===	Equal to and equal type	"2" === 2	false
!==	Not '==='	"2" !== 2	true
!=	Not '=='	"2" != 2	false
>	Greater then	"3" > 2	true
>=	Greater then or equal to	"3" >= 3	true
<	Less then	"4" < "3"	false
<=	Less then or equal to	"4" <= "4"	true

* Comparing a string to a number, will always convert the string to a number. If the string cannot convert to a number, it will become 'NaN', which will always result in the false

Logical operators

- Python has the word 'and', 'or', and 'not'
- JS instead has '&&', '||' and '!'
- e.g. this 'x < y and (a == b or not check())'
- Becomes 'x < y && (a == b || !check())'


```
1  for (let i = 0; i < 10; i++) {  
2    |    console.log(i);  
3  }
```

Template Strings

```
1  function greet(name) {  
2    |    return `Hello, ${name}`;  
3  }  
4  
5  console.log(greet("Alice"));
```

- In python you have 'f-strings', in JS you have 'template strings'
- These are done by putting them in back ticks ``
- And then expressions run within any '\${ }'

String Props

- Instead of python's odd method of getting length, JS strings have the property `'.length'`
- Just like python's `'x[1:7]'` slicing, JS has the `'slice'` method which has the parameters `'start (inclusive)'` and `'end (exclusive)'`
- You can check if a string contains a substring with `'.includes(x)'`

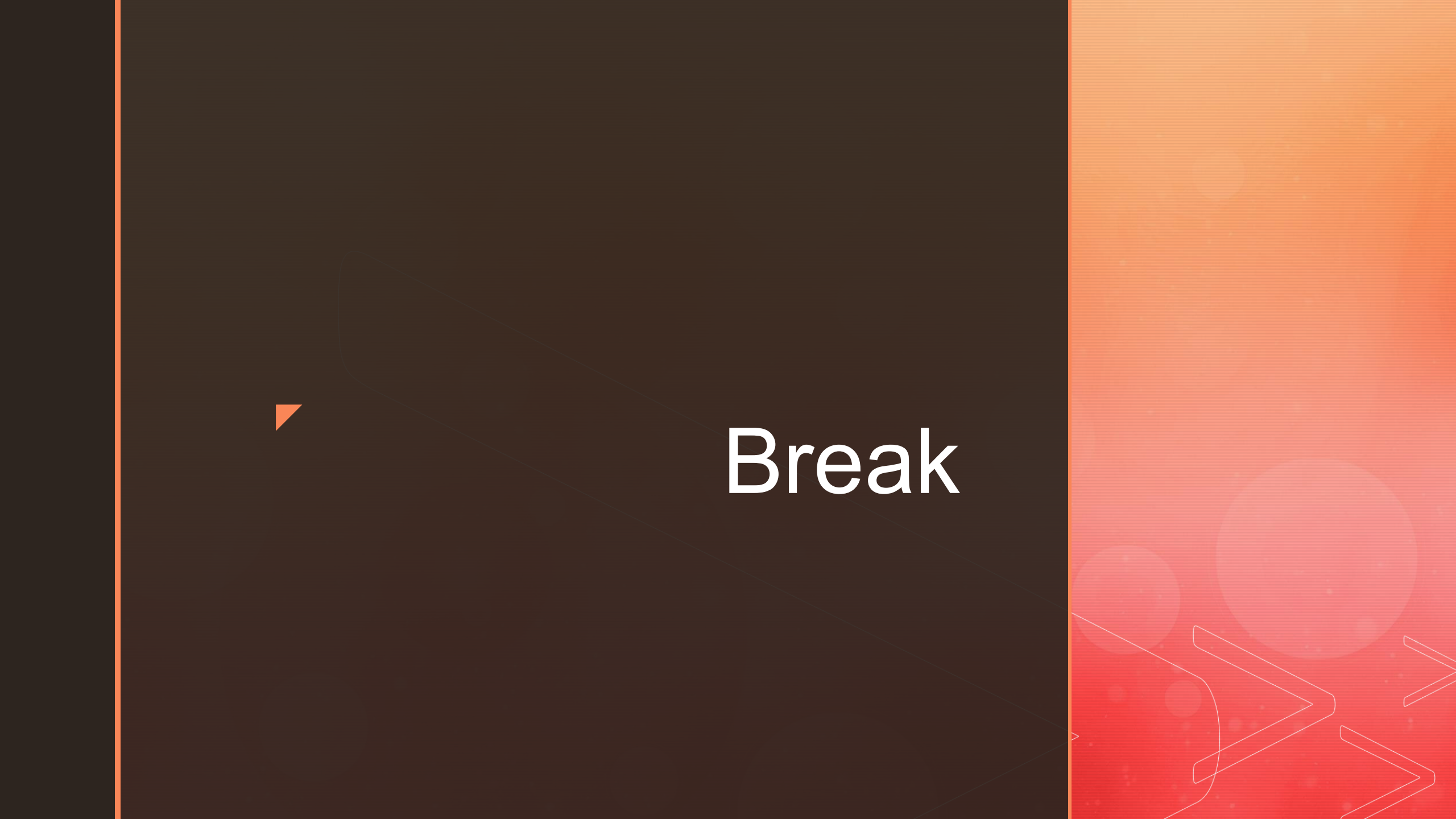


Numbers

- To check if a number is 'NaN' the function 'isNaN(x)' can be used

Type Conversions

- If you want to convert the type of a variable, then just like In python, you just call a function called that type name
- But the function has a capital first letter
- e.g. `Number("17") === 17`
- `String(17) === "17"`
- `Boolean(17) === true`



Break

The DOM

- So far, the javascript we have gone over is useless as it can only print things to the console
- Now it is time to cover how these scripts interact with the webpage to control stuff

The DOM

- Webpages use a 'Document Object Model' (DOM) that is a tree of nodes starting from the 'Document' node
- To interact with this, the typical way in javascript is to grab the webpage document, and then query it for a certain element within that

Setting Text

```
1 // Get the paragraph node by its id
2 const p = document.getElementById("description");
3
4 // Change the text within this paragraph element
5 p.innerText = "My new description";
```

- In JS 'document.getElementById()' can be used to grab an HTML node based on the 'id' given to it
- 'innerText' can then be used to edit the text of such an element

Setting HTML

```
1  // Get the paragraph node by its id
2  const p = document.getElementById("description");
3
4  // Change the text within this paragraph element
5  p.innerHTML += "<br><button>New Button</button>";
```

- While before only text could be set, if we want to insert HTML inside, then the '.innerHTML' can be used
- This can ofcourse also be read from, and appended to

Setting attributes

```
1  // Get the paragraph node by its id
2  const b = document.getElementById("funButton");
3
4  // Disable the button
5  b.disabled = true;
```

- Once any node is grabbed, its properties/attributes can be set
- e.g. the 'disabled' attribute of a button
- Just as these attributes can be set, they can also be read

Setting CSS

```
1 // Get the paragraph node by its id
2 const p = document.getElementById("description");
3
4 // Make the text red
5 p.style.color = "red";
```

- The CSS of an element is really just an attribute, so it can also be set with JS

Calling Functions

```
<button id="funButton" onclick="yes()">Click Me!</button>
```

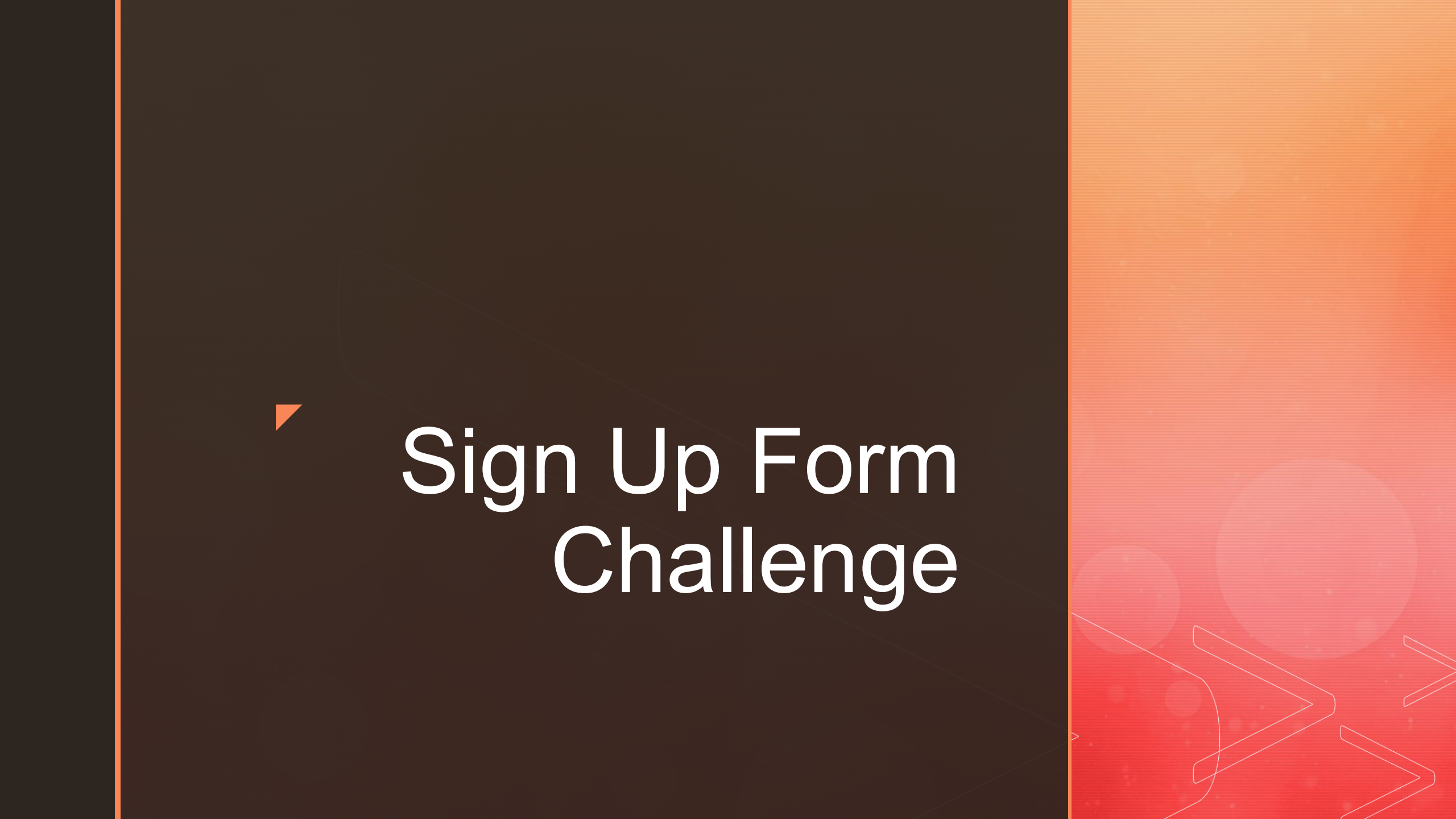
```
1  function yes() {  
2      const p = document.getElementById("description");  
3      current_value = Number(p.innerText);  
4      if (isNaN(current_value)) {  
5          p.innerText = 0;  
6      } else {  
7          p.innerText = current_value + 1;  
8      }  
9  }
```

- To get a button to have functionality, the 'onclick' attribute of the button can be set to be a function

Input Value

```
1  function yes() {  
2      const inp = document.getElementById("inputing").value;  
3      const p = document.getElementById("description");  
4  
5      if (inp > 18) {  
6          p.innerText = "18+";  
7      } else if (isNaN(Number(inp))) {  
8          p.innerText = "NaN";  
9      } else {  
10         p.innerText = "< 18"  
11     }  
12 }
```

- To get the value of an 'input' element in JS, you get the node, then read the '.value' element



Sign Up Form Challenge

this

```
1  const description = document.getElementById("description");
2
3  function clicked(caller) {
4      |   buttonId = caller.innerText;
5      |   description.innerText = `You clicked button ${buttonId}!`;
6  }
```

- JS has the 'this' keyword which can be used by objects to refer to themselves
- An example would be for html buttons to do it, so the function they call knows which button called the function

Node Traversal

- In JS when you have a node, you can get its parent node with 'parentElement'
- You can also get the children on a node through '.children' which returns an array of all the children in the order they are defined in the code
- This means you can get the siblings to a node through '.parentElement.children'

interval

```
1  let int;  
2  
3  function print() {  
4    |   console.log(1);  
5  }  
6  
7  function set() {  
8    |   if (int == undefined) {  
9    |     |   int = setInterval(print, 500);  
10   |   } else {  
11   |     |   clearInterval(int);  
12   |     |   int = undefined;  
13   |   }  
14 }
```

- Intervals can be set, to run functions ever so often
- You use 'setInterval', passing in the function to call, and how many milliseconds there should be between calls
- This function returns the interval ID, which is used to clear the interval with 'clearInterval'

Timeout

```
1  const messageHolder = document.getElementById("description");
2
3  function remove() {
4      messageHolder.hidden = true;
5  }
6
7  function msg () {
8      messageHolder.innerText = "Don't press me!";
9      messageHolder.hidden = false;
10
11      setTimeout(remove, 2000);
12  }
```

- If you just want the function to run once, after a certain amount of time then you can use 'setTimeout' passing in the function, and the time until it will run

Lambda Functions

- You might have noticed with those two, that for simple/one off functions it is useless to define the function then just pass it into something else
- Instead an anonymous/lambda function can be created
- With this instead of defining a function for the code, the function object is created and immediately returned without being added as a local or global variable.

Timeout Lambda

```
1  const messageHolder = document.getElementById("description");
2
3  function msg () {
4      messageHolder.innerText = "Don't press me!";
5      messageHolder.hidden = false;
6
7      setTimeout(function () {
8          messageHolder.hidden = true;
9      }, 2000);
10 }
```

- Lambda functions can be defined by creating a function but leaving out the name of it

Arrow notation

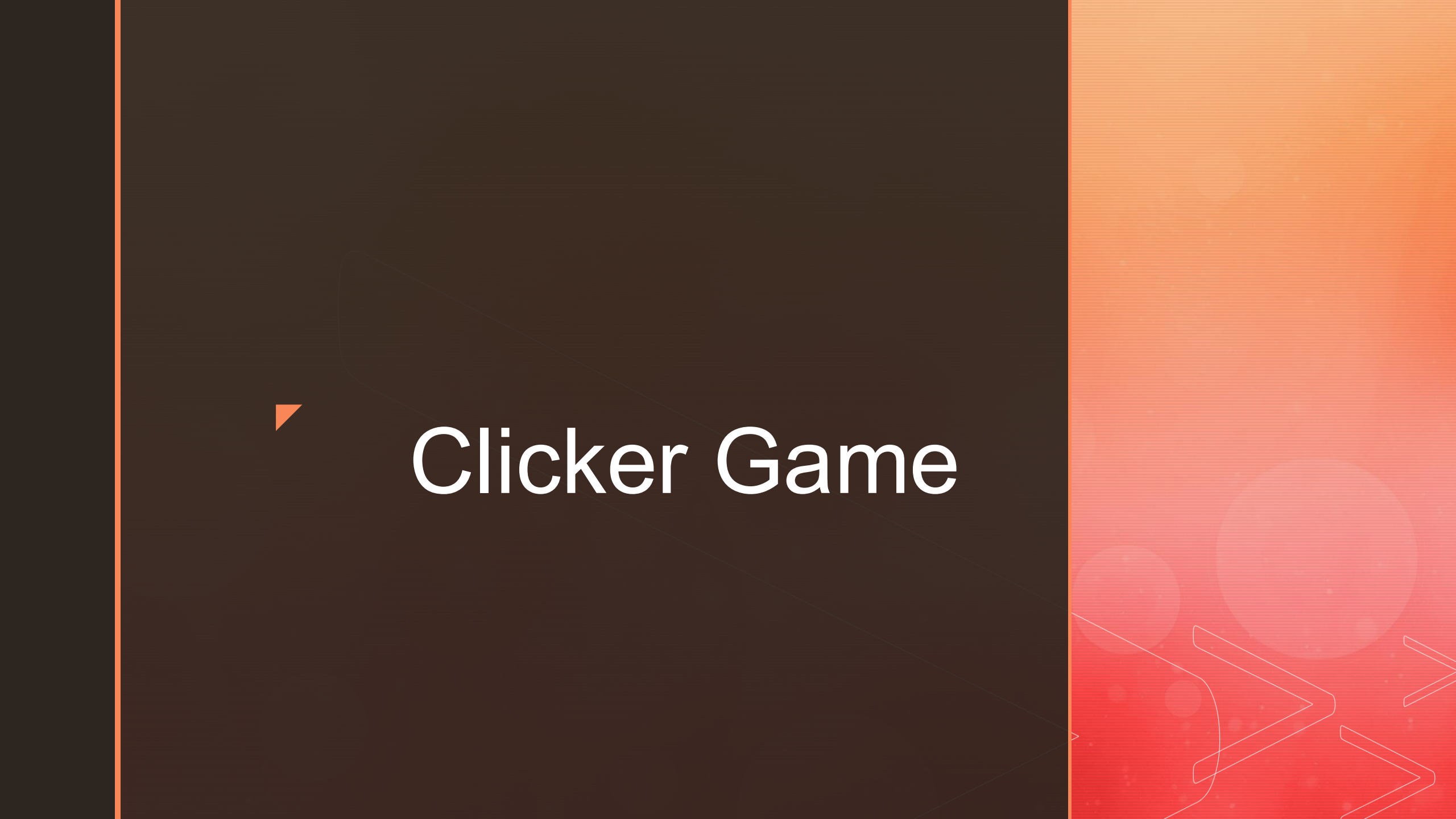
```
1  const messageHolder = document.getElementById("description");
2
3  function msg () {
4      messageHolder.innerText = "Don't press me!";
5      messageHolder.hidden = false;
6
7      setTimeout(() => {
8          messageHolder.hidden = true;
9      }, 2000);
10 }
```

- There is another way to make a lambda function called arrow notation
- This lets you use an '=>' instead of the word 'function'

Arrow Notation Cont.

```
1  const messageHolder = document.getElementById("description");
2
3  function msg () {
4      messageHolder.innerText = "Don't press me!";
5      messageHolder.hidden = false;
6
7      setTimeout(() => messageHolder.hidden = true, 2000);
8  }
```

- For arrow notation if the function is just one line, then the brackets and semi-colon are not needed



Clicker Game

Arrays

```
1  let arr = [7, 3, 9, 5];  
2  
3  console.log(arr.length);  
4  console.log(arr[2]);
```

- Python has lists, JS has arrays
- Accessing and creating them is the exact same

```
1  let arr = [7, 3, 9, 5];
2
3  // Add to end
4  arr.push(7);
5  console.log(arr);
6
7  // Remove from end
8  arr.pop();
9  console.log(arr);
10
11 // Add to start
12 arr.unshift(12);
13 console.log(arr);
14
15 // Remove from start
16 arr.shift();
17 console.log(arr);
```

Array Manipulation

- The four main function to be used are 'push', 'pop', 'shift', and 'unshift'

join

```
1  let arr = [7, 3, 9, 5];  
2  
3  arr2 = arr.join(":");  
4  
5  console.log(arr2);
```

- Just like in python, there is 'arr.join(str)'

map

```
1  let arr = [7, 3, 9, 5];  
2  
3  let arr2 = arr.map((num) => num * 2);  
4  
5  console.log(arr2);
```

- This method loops through every element in an array, gets passed a function, and returns a new array which has the result of all the functions

filter

```
1  let arr = [7, 4, 9, 8];  
2  
3  let arr2 = arr.filter((num) => num % 2 == 0);  
4  
5  console.log(arr2);
```

- This is a method that returns a new array which only contains elements if the function that the method was passed returned true for that item

reduce

```
1 let arr = [7, 4, 9, 8];  
2  
3 let val = arr.reduce((num, sum) => sum += num);  
4 💡  
5 console.log(val);
```

- This has an accumulator, and runs through the entire array, editing the accumulator which is saved from each call

sort

```
1  let arr = [7, 4, 9, 8];  
2  
3  arr.sort((a, b) => a - b);  
4  
5  console.log(arr);
```

- This lets you pass in a function, which if it returns '1' swaps the order of the two elements, '-1' does not

promise

- Javascript is a synchronous language
- As such, if any asynchronous task needs to take place such as an API request, a promise is made
- A promise is either 'pending' when it is still waiting to be completed, 'fulfilled' where it succeeded, or 'rejected' where it failed

then

- As these promises can be fulfilled at any time, instead of periodically checking if it is fulfilled, JS provides a way to give the promise a function that it will run when it succeeds
- This function is the '.then' method
- A promise can have the '.then' method called on it, where a function is passed in that will be called when the promise is fulfilled, and will be given an argument of the promise return data

catch

- These promises can also fail, and so there is an alternative to '.then' which runs if the promise ever fails
- It is called '.catch' and is used in the exact same way

fetch

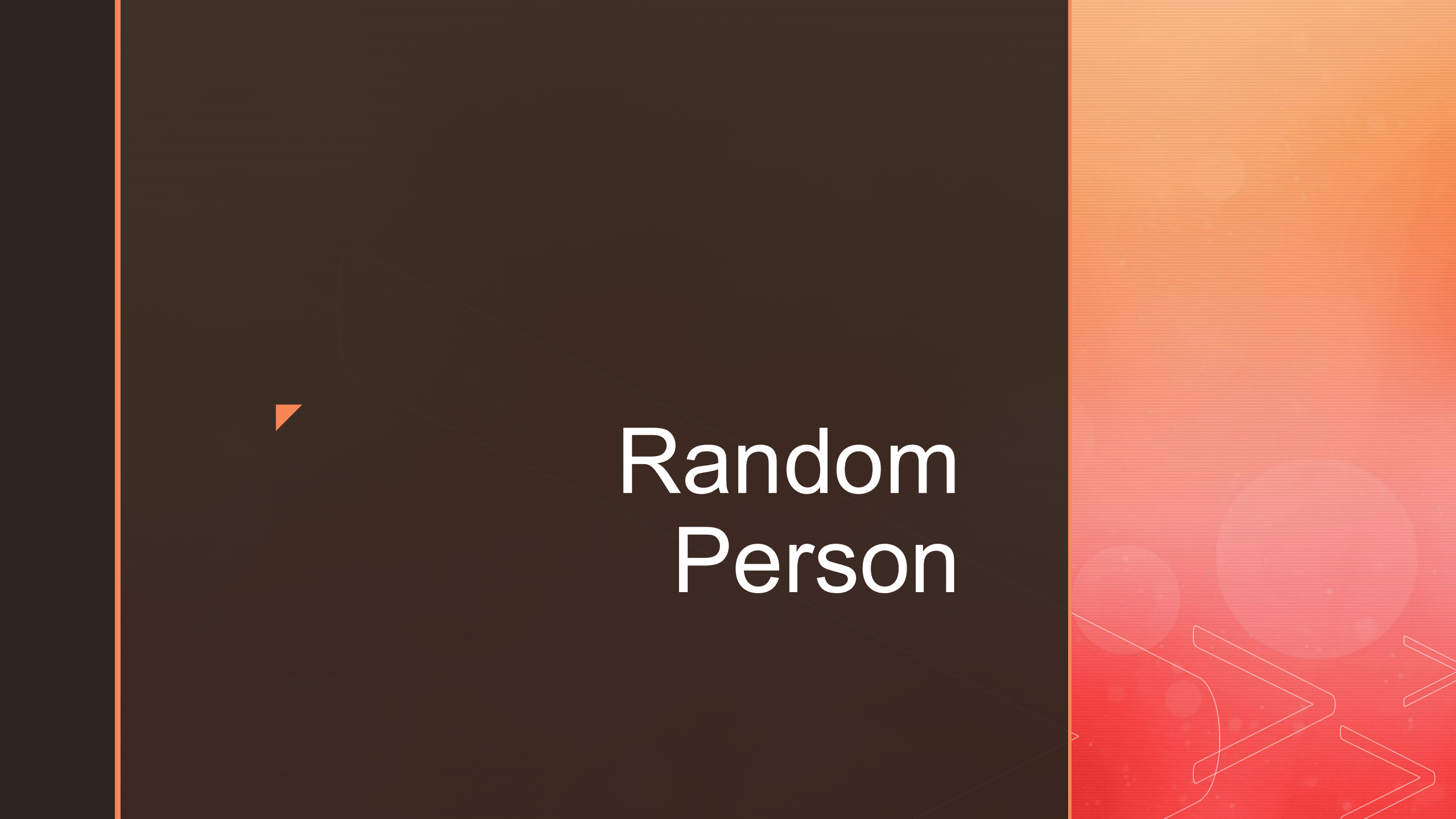
```
1  fetch("https://catfact.ninja/fact/")
2    .then((response) => {
3      // Check if the request got a reply,
4      // but the reply was an error message
5      if (!response.ok) {
6        throw new Error(`Status Error ${response.status}`);
7      }
8      return response.text();
9    })
10   .catch((err) => {
11     console.log(err.message);
12   })
13   .then((message) => {
14     console.log(message);
15   })
```

- The most common way you will get a promise is with the 'fetch' function
- This will fetch the url passed in, and return a promise which will fulfill with the returned data from url

response.json

```
1 function reloadFact() {
2   fetch("https://catfact.ninja/fact/")
3   .then((response) => {
4     // Check if the request got a reply,
5     // but the reply was an error message
6     if (!response.ok) {
7       throw new Error(`Status Error ${response.status}`);
8     }
9     return response.json();
10  })
11  .catch((err) => {
12    console.log(err.message);
13  })
14  .then((message) => {
15    const p = document.getElementById("description");
16    p.innerText = message.fact;
17  });
18 }
```

- response.json is used to parse the json code of a url request into an object, that has attributes



Random Person